

Electronic Signature (eSign) Document Management System

Objective	Build a React-based full-stack web application that allows users to upload PDF documents, configure electronic signature placeholders for different signer roles, and manage documents through a document library. The application should provide an intuitive drag-and-drop interface for placing signature fields on PDF pages and support editing previously configured documents.
Why it matters	Electronic Signature solutions eliminate paper-based document signing and improve workflow efficiency. This project introduces practical full-stack concepts such as file upload and validation, PDF rendering, drag-and-drop interactions, state management, CRUD operations, responsive UI design, backend API integration, and database persistence. It closely resembles real-world enterprise CRM/eSign functionality.
Functional Requirements	Document List Screen: Display a grid/table with Document Name, Uploaded By, Uploaded Date, Number of Signers, Status, and Actions (Edit). Include an Upload Document button in the top toolbar. Upload Document: Accept only PDF files, enforce a 5 MB maximum size, show validation messages, and open the configuration modal after a valid upload. Configure Document Modal: Provide a left panel for document details and draggable signer roles such as Buyer, Seller, Manager, Approver, and Witness. Render the uploaded PDF on the right with scrollable pages, good rendering quality, zoom controls, and jump-to-page support. Drag & Drop Signature Fields: Allow users to drag signer cards, drop them anywhere on the PDF, place multiple signers, and maintain signer type, page number, X coordinate, and Y coordinate. Save/Edit Document: Persist PDF metadata, signer positions, signer information, and document name. Support loading saved documents, repositioning fields, adding/removing placeholders, and saving changes.
Getting Started	• Start by creating the React application structure and deciding the UI component library: Material UI or Ant Design. • Implement PDF upload with client-side validation for file type and size before moving to backend persistence. • Build the PDF rendering screen early so drag-and-drop positioning can be tested against real PDF pages. • Define the backend API contracts for upload, save configuration, document list, edit document, and update signer positions before full implementation.
Tools	Frontend: React, TypeScript Backend: Any Database: Any <i>You can choose Backend tool as per your choice.</i>
Duration	2 Weeks
Team & Responsibilities	• You own the full-stack implementation, including the React UI, backend APIs, database schema, and integration flow. • Mentor guidance should be used for architecture review, API design, drag-and-drop positioning approach, and testing strategy. • You should document setup steps, assumptions, validation rules, API contracts, database design, and known limitations in the README. • The final demo should clearly show upload, PDF rendering, signer placement, save, document listing, and edit flows.

Deliverables	1. A working full-stack application for uploading PDFs, configuring signer placeholders, saving documents, listing documents, and editing saved configurations. 2. Source code in a GitHub repository with a clear folder structure and README setup instructions. 3. API documentation covering upload, document list, save configuration, edit document, and update operations. 4. Database schema documentation for Document and Signer entities, including key fields and relationships. 5. Test cases covering PDF validation, required document name, at least one signer, save flow, and edit flow. 6. A one-page architecture diagram showing frontend, backend, storage, database, and API flow.
---------------------	---